IRONHACK AI ENGINEERING BOOTCAMP · FINAL PROJECT 9 · PROJECT REPORT

The Factory Floor

An industrial maintenance copilot for Siemens electric motors and variable-frequency drives: a technician asks in their own language, optionally with a photo, and gets an answer grounded in the real manufacturer manuals, with page citations, safety precautions first, and that machine's own repair history.

Marcelo Correia · 5 September 2026 · github.com/mcorreia10/Factory_Floor_Chatbot (public, CI passing, MIT)

3,679	18 → 100%	33 → 5.9%	183
pages from 11 official Siemens manuals, 12,301 chunks	fault-code queries that bring the defining page	safety-ordering failures, raw prompt → delivered answer	automated tests, no API key, run on every push

1 Problem and objective

Manufacturing loses large sums to unplanned downtime, and the deeper problem is human: experienced maintenance technicians retire faster than they are replaced, and their troubleshooting knowledge leaves with them. The industry's answer is the "connected worker": an assistant that puts manuals, safety information and maintenance history in front of the technician standing at the machine.

Concretely: a SINAMICS drive is flashing `F30021`. The answer is somewhere in 3,679 pages spread across operating, list and function manuals. Three things actually happen on the floor: the technician already knows it, calls someone, or guesses. And very often the previous shift already solved this exact fault on this exact machine, and nobody wrote it down where the next person would find it.

Objective. Let a technician verify any answer against the manual page it came from in under ten seconds, get the safety precautions before any physical step every single time, and see what was done the last time this fault hit this machine. The project also has to meet the bootcamp's seven core requirements (RAG with citations, agents, tracing, a multimodal component, a 30+ task evaluation with baseline benchmarking, a deployed web app, and a presentation), and the project-specific evaluation: defect-classification accuracy, 30+ troubleshooting scenarios, and a safety audit whose number is reported honestly.

> **Scope, stated plainly.** This is decision support for a trained technician built as a portfolio project. It is not a certified industrial safety system. The residual failure rates are published in section 5, not tuned until they looked good.

2 Approach and architecture

Retrieval, not a fine-tuned model. There is no target column in this project: the knowledge is the manuals themselves. Retrieval-augmented generation keeps the system traceable (every claim has a page to open), always current (replace the PDF and re-index), and unable to invent a fault code (it cites

only what came back from the index). The bootcamp spec advises against fine-tuning for the same reasons.

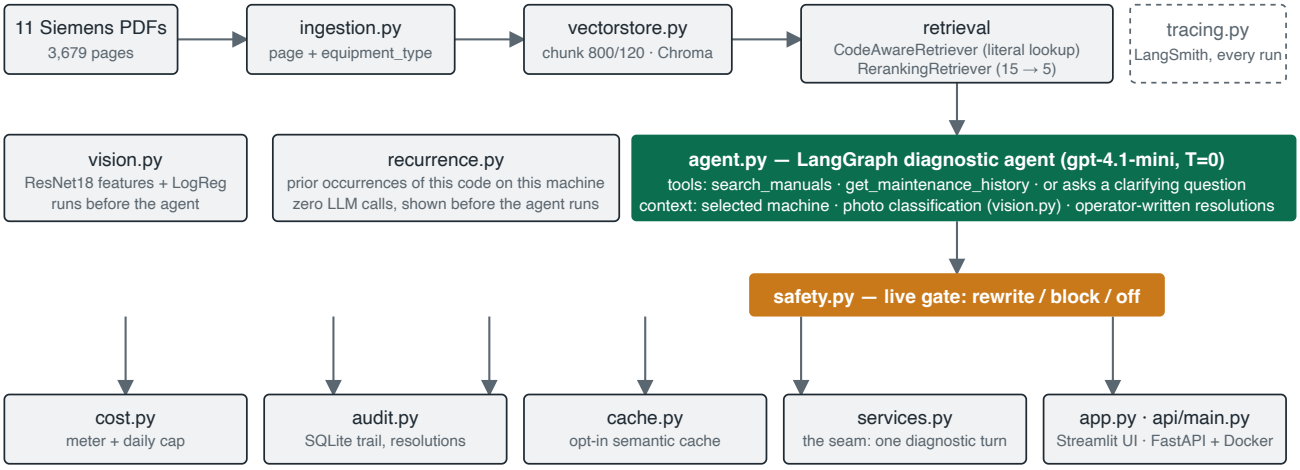

Figure 1. The pipeline. `services.run_diagnostic()` is the single seam every layer wraps: the Streamlit app reads widgets and renders, the FastAPI endpoint does JSON, and neither owns any pipeline logic.

Table 1. Technology used

Layer	Choice
Retrieval	LangChain 1.3 · Chroma (persistent) · OpenAI `text-embedding-3-small` · chunk 800 / overlap 120 · k = 5 · optional LLM reranker · literal fault-code lookup via Chroma's full-text filter
Reasoning	`gpt-4.1-mini` at temperature 0 · `langchain.agents.create_agent` (LangGraph) · two tools · session memory through history-aware question reformulation
Vision	torchvision ResNet18, frozen, as a feature extractor + logistic regression · curated 4-category MVTec AD subset
Observability	LangSmith: every run tagged and routed to a dedicated project, permalink per run
Serving	Streamlit UI (5 languages) · FastAPI proof (`/health` , `POST /diagnose`) · Dockerfile
Quality	183 pytest tests (unit + integration, fake models, no API key) · ruff · GitHub Actions on Python 3.12 and 3.13

3 Data

Corpus: 11 official Siemens manuals, 3,679 pages. Six drive manuals (SINAMICS G120 / G120C / CU240B-2 / CU240E-2) and five motor manuals (SIMOTICS GP / SD / DP / 1LE1 / 1LE7), across five document types: operating instructions, list (fault) manuals, function manuals, an engineering manual and a catalog. Roughly 2,766 drive pages to 913 motor pages. Every row of `manual_sources.csv` carries an explicit `equipment_type` that ingestion reads instead of guessing from the filename.

Fault codes: 368 real codes (206 faults, 162 alarms) extracted by a regex scan of every page of the raw PDF text, with the defining line scored rather than the first mention. An earlier draft used plausible-looking codes such as `F0051` that simply do not exist in these manuals; the scan replaced them. Electric motors have no numeric fault-code system at all, confirmed by the same scan returning zero matches across all five motor manuals.

Fleet: 20 simulated assets (10 motors, 10 drives) with 77 dated fault, repair and preventive events, generated once with a fixed seed and committed as static data. The spec asks for recurring faults for the agent to discover; the data has them.

Defect images: MVTec AD, curated. The dataset ships 15 object categories, most irrelevant to a motor or a drive. Four were kept as physically plausible on or around the equipment: cable, metal nut, screw, transistor. 1,502 images, 1,126 for training and 376 held out, with a shared six-label vocabulary (good, scratch, deformation, structural damage, contamination, other).

4 The system, component by component

4.1 Retrieval pipeline, with every parameter measured

Pages are split at natural breaks into 800-character chunks with 120 overlap, embedded and stored in Chroma with page and equipment-type metadata. Selecting a machine in the UI filters retrieval to that machine's equipment family. The reranker widens the pool to 15 candidates and lets one LLM call order them, keeping five. Section 5.4 reports the ablation behind each of those numbers; the short version is that 400-character chunks split a fault entry's *Cause* from its *Remedy*, and the reranker buys ordering rather than coverage.

4.2 The biggest challenge: fault codes are labels, not concepts

An embedding model has nothing to represent when the query is `F30021`. Every page of a list manual has the same shape (*Power unit: … Cause: … Remedy: …*), so a bare code landed on whichever fault page was semantically nearest, and with no relevance floor the retriever always returned five confident neighbours. Measured over the 17 verified codes, querying with the code alone: the context contained the code somewhere in 82% of cases, but brought the page that *defines* it in only 18%. Worse, an unknown code produced a real, well-cited answer about a different fault.

The fix was already in the stack. Chroma's full-text filter (`where_document={"$contains": code}`) does exact substring matching with no embedding call, no new dependency and no re-indexing. Codes are now *looked up*, not searched: a regex detects the code, the literal occurrences are scored so the definition wins over cross-references (the first literal hit for `F30021` is a "See also" on page 62; the definition is on page 908), the exact hits are pinned to the top and never passed to the reranker. Result: 100% on both columns. The guardrail matters more than the accuracy: zero literal hits is an unambiguous signal, so `F99999` now refuses instead of describing another fault, and a display typo like `F30021` (letter O for zero) normalises onto a real code and the app *asks* which was meant rather than silently correcting.

4.3 The agent, and three kinds of memory

The agent (`create_agent` over LangGraph) decides per turn whether it needs evidence. It can call `search_manuals` as many times as it wants, call `get_maintenance_history` for the selected asset, or call nothing and ask one clarifying question. A photo is classified before the agent reasons and injected as context, so the agent decides what to do with the result. The system prompt requires a manual search before any diagnostic claim, requires searching with the operator's own symptom words rather than a jargon paraphrase (rephrasing measurably retrieved worse), and never translates codes or parameter numbers, because the operator has to recognise them on the equipment display.

- **The conversation.** An elliptical follow-up ("and the other one?") is rewritten into a standalone question before retrieval; turn one is byte-identical to a fresh question.

- **The machine.** The history tool returns the asset's static events plus the resolutions operators wrote inside the app, capped at 20 rows, and flagged to the model as unverified field notes, never citable as manual content.
- **The shift before yours.** If the question carries a fault code this machine has already seen, a panel of prior occurrences appears *before* any model call, computed from records with zero LLM calls.

> **The panel reports and refuses to recommend.** On the real data, VFD-04's `F30805` was answered twice with a power-unit module replacement (and came back), then three times with a power cycle. A frequency ranking would recommend the power cycle three votes to two, that is, recommend masking a recurring hardware fault. So `recurrence.py` counts, sorts and dates, and a test fails if anyone adds a recommendation key. An `outcome` field is now collected on every resolution so that, after months of use, effectiveness can be ranked on evidence.

4.4 The safety-first contract, enforced live

Rule 6 of the agent prompt outranks every other rule: if the answer contains any step a person physically performs, it must begin with a safety-precautions section. A prompt rule alone is not reliable (section 5.3), so the finished answer passes through a gate before the operator sees it: a cheap keyword pre-check, then an LLM judge with structured output. A failing answer is rewritten once, re-checked, and verified to have kept every citation and fault code; if it still fails it is held and a fixed fallback is shown. Modes: `off`, `rewrite` (default), `block`. The gate's own LLM calls are metered against the daily spend cap.

4.5 The multimodal component

Frozen ResNet18 features with a logistic-regression head, no fine-tuning: with about 1,500 images in four categories, fine-tuning overfits before it helps. The classification is surfaced to the operator as a condition plus grounded next steps through the agent, not as a bare label, because a label alone is useless on the floor. The classifier's benchmark is in section 5.1.

5 Evaluation and benchmarks

The spec asks for three measurements: defect-classification accuracy on a held-out split, at least thirty troubleshooting scenarios with an expected root cause and manual section, and a safety audit. All three exist, plus a retrieval ablation and an agent-versus-baseline comparison. Every number below is the notebook's printed output as of 4 September 2026 (`notebooks/06`, `10`, `11`).

5.1 Vision: the interesting number is the last one

Approach (376 held-out images, 4 categories)	Accuracy
Trained classifier: frozen ResNet18 features + logistic regression	**82.2%**
Majority-class baseline (always predict "good")	77.1%
Zero-shot LLM (`gpt-4.1-mini` on the image)	**42.5%**

The zero-shot model is worse than doing nothing: it invents defects in photographs of undamaged parts (7% recall on the "good" class). That is a real failure-analysis finding, and it is why a trained classifier sits in front of the LLM.

5.2 Agent versus single-call baseline over 37 scenarios

`eval_scenarios.csv` holds 37 troubleshooting scenarios (17 drive fault codes, 10 motor shop-floor complaints, 4 motor manual questions, 3 drive general, 3 general), each with an expected root cause and a manual evidence phrase individually verified against the vector store. The evidence phrase is always confirmed absent from the question, so nothing can score by echoing the prompt. The baseline is the same retriever with one search and one answer, no tools, no history, no follow-up question.

Table 2. Same manuals, same retriever, same 37 questions

Metric	Baseline (single-call RAG)	Agent
Root-cause accuracy	73.0%	**75.7%**
Evidence accuracy (verified manual phrase present)	45.9%	**54.1%**
Source-family accuracy (right manual)	94.1%	94.1%
Machine-history awareness	0% (structural: no history tool)	cites the real event date
Conflicting signals (photo: structural damage; text: "a tiny cosmetic scuff")	**"likely safe to continue running"**	**asks a clarifying question**
Mean latency · mean tool calls	3.5 s · 0	6.7 s · 1.0

The keyword rows are close; the structural rows are not. The agent's evidence-accuracy lead came from one prompt change made on 4 September: without it, the agent rephrased colloquial symptoms into technical synonyms before searching, which collapsed motor-scenario evidence accuracy to about 7%. Searching with the operator's own words brought it to 71% on those scenarios.

5.3 The safety audit: the distinctive number

Every evaluation answer is audited twice, by an LLM judge and by an independent deterministic regex check (agreement 94.6% on agent answers, 78.4% on baseline answers, published as the confidence in the judge). The baseline prompt deliberately does *not* carry the safety rule: if both pipelines had it, the comparison would measure "does GPT follow instructions" twice instead of whether the architecture matters.

Table 3. Answers that recommended a physical action but did not put precautions first

Pipeline	Answers with an action	Ordering failures	Failure rate
Baseline, no safety rule	34	31	**91.2%**
Agent, raw prompt (rule 6 active)	33	11	33.3%
Agent, delivered answer (after the live gate)	34	2	**5.9%**

Zero is the target and the number is reported as measured. The gate rewrote 10 answers; the two that survive (`GEN01` , `GEN02`) are named in the notebook rather than tuned away. The raw 33.3% is the honest instruction-following ceiling of a prompt rule, and it is exactly why the post-hoc audit became a live blocking gate.

5.4 Retrieval ablation: one axis at a time, scored with zero LLM calls

Table 4. Hit-rate@5 and MRR over the same 37 scenarios (full write-up: `Retrieval_Benchmark_Report.pdf`)

Axis	Variants	Hit-rate	MRR	Reading
Chunk size	400 · **800 (current)** · 1600	64.9 · **78.4** · 81.1%	0.433 · **0.596** · 0.610	At 400 the fault-code category collapses from 82% to 59%: Cause and Remedy end up in different chunks
Search strategy	**similarity** · mmr · score threshold	**81.1** · 73.0 · invalid	**0.610** · 0.591 · —	Threshold reported as an invalid measurement: the collection is in L2 space, so relevance scores fell outside 0–1
Reranker	off · **on**	81.1 · **86.5%**	0.610 · **0.695**	Same hit-rate as plain k=10 (MRR 0.619) with clearly better order on half the context; batch latency 7 s → 43 s
Top-k	3 · **5** · 10	75.7 · **81.1** · 86.5%	barely moves	Hit-rate rises mechanically with k; ordering does not

5.5 Observability

Every run is traced end to end in LangSmith: model calls, retrievals, tool use, in order. Over the project's lifetime: 1,620 runs recorded, 0% ended in an exception, median latency 1.71 s, P99 8.98 s. Zero crashes is not zero mistakes: tracing flags exceptions, and a confidently wrong answer still counts as a success there. That is what the evaluation above is for.

5.6 Measuring honestly: three times a number lied

- **The baseline scored on a metric it structurally cannot pass.** The first history-awareness metric matched either the event date or the fault code; since every question names its own code, the baseline "referenced history" without having a history tool. Fixed by matching only the date. Baseline now scores a clean 0%, as the structure predicts.
- **The question contained its own answer.** Every drive scenario names the code and the symptom, so a naive keyword check rewarded echoing the prompt. Evidence keywords are now verified absent from the question, enforced by an assertion on every row.
- **The safety gate was off in four of the five languages.** The cheap shortcut that skips the judge when no physical action is recommended used a regex of English verbs, so Portuguese, French, Spanish and German answers never reached the judge, while the gate reported itself active. Found by chasing a user symptom ("why is the Portuguese answer nothing like the English one?"), not by reading the code.

Two features were built, measured and **deliberately reverted**, and written up rather than quietly deleted: a real thermal-image motor dataset that trained to 98.9% but whose 369 images were only 11 independent scenes of near-identical frames (no split could measure generalisation), and a component-identity classifier at 100% whose categories each had their own capture background, so transfer to a real photo was never established.

6 Production concerns, built in

Everything below was added after the bootcamp requirements were met, to take the system from a working demo to something that could survive contact with a deployment. All of it is opt-in and off by default: a fresh clone with only an OpenAI key behaves exactly like the original RAG app. Each knob is a `FACTORY_FLOOR_*` environment variable with a safe default.

Concern	How it is handled
Configuration and secrets	Typed frozen `Settings`; a `get_secret()` seam with `env` today and vault backends designed; the key never appears in a `repr`
Cost	Per-token metering, per-session total, a daily ledger and a hard spend cap that blocks the turn before it runs, including the safety gate's own calls
Auditability	SQLite (WAL) trail of every recommendation, its sources, tools, cost and operator; operator-written resolutions with outcome; a CMMS-export demo and an email hand-off
Identity	Operator sign-in with PBKDF2-hashed PINs, managed by a CLI, never hand-edited; every action tied to a person
Caching	Opt-in semantic answer cache in cosine space; exact-match only for fault-code questions, never a near miss
Multi-tenancy	Collection-per-tenant seam with a written threat model; one deployment, several plants, kept separate
Serving	The Streamlit UI and a FastAPI endpoint share one service function; Dockerfile for the API, data mounted at runtime
Tests and CI	183 tests with fake models and a tiny real Chroma store, no key, no cost; ruff; GitHub Actions on Python 3.12 and 3.13

7 Limitations and what comes next

Known limits

- **Evidence accuracy at 54%** is the weakest number. The keyword scorer is a proxy: a correct answer worded differently counts as a miss.
- **Two answers still fail the safety ordering after the gate** (5.9%). The judge is the same model family as the generator; the regex cross-check is the mitigation, not a cure.
- **No relevance floor.** Off-topic questions still show five sources. A threshold cannot be tested until the store is rebuilt in cosine space.
- **Vision data is not domain data.** MVTec parts, not worn bearings or burnt windings; transfer to a photo inside a cabinet is not established.
- **No safety-data-sheet corpus**, so precautions cover what the manuals say, not lubricants and chemicals. Diagrams and nameplates are lost by text-only PDF extraction. The corpus is 3:1 drive to motor.

What I would build next

- Rebuild the vector store in cosine space and measure a real relevance threshold.
- Replace keyword scoring with an LLM-as-judge rubric, calibrated against a human-labelled sample.
- Assemble real motor and drive defect photographs; the neighbouring-frame test from the abandoned thermal experiment is the reusable part.
- Ingest safety data sheets for the lubricants and cleaning agents used on the equipment.
- Audio diagnostics: a real dataset exists (University of Ottawa, an induction motor driven by a real drive, 128 recordings) and was researched and deferred because it needs real preprocessing.
- A configuration knob for the reranker, the most optional LLM call on the critical path.

A Appendix

A.1 Core requirements, and where each one lives

Requirement	Where
RAG with cited sources, vector database	`rag.py`, `fault_codes.py`, Chroma; every answer cites file and page
Agentic architecture with tools and memory	`agent.py`: two tools, clarifying questions, conversation and machine memory
Tracing and observability	`tracing.py`, LangSmith project with a permalink per run; `notebooks/08`
Multimodal component	`vision.py`, defect classification on MVTec AD; `notebooks/06`
Evaluation (30+ tasks) with baseline benchmarking	37 scenarios, agent vs baseline, safety audit, retrieval ablation; `notebooks/10`, `11`
Deployed web application	Streamlit app; FastAPI + Dockerfile as the serving proof
Final presentation	`docs/Final_Presentation.pptx`, delivered 5 September 2026 with a live demo

A.2 The 17 verified fault codes used in the fleet history and the evaluation set

F30001 overcurrent · F30002 DC link overvoltage · F30003 DC link undervoltage · F30004 heat-sink overtemperature · F30011 line phase failure · F30021 ground fault · F30035 air-intake overtemperature · F30037 rectifier overtemperature · F30059 internal fan faulty · F30805 EEPROM checksum error · F30950 internal software error · F07011 motor overtemperature · F07016 motor temperature sensor fault · F07801 motor overcurrent · F07807 short-circuit at motor terminals · F07860 external fault 1 · F07901 motor overspeed. Each was verified individually against the real Cause/Remedy text in the list manuals.

A.3 Running it

```
pip install -r requirements.txt && cp .env.example .env     # add OPENAI_API_KEY
python download_manuals.py && python download_defect_images.py
# run notebooks/01 and 02 once to build data/vectorstore/, then:
make app      # Streamlit UI        make test   # 183 tests, no key, no cost
uvicorn api.main:app --reload       # or the API: /health, POST /diagnose
```

The Factory Floor · Marcelo Correia · Ironhack AI Engineering Bootcamp, final project · September 2026 · MIT licence · Source, notebooks, evaluation data and this report: github.com/mcorreia10/Factory_Floor_Chatbot